

Shomer.AI — Design Index & Architectural Principles

Status: Draft for Meeting 4 Last updated: 2026-05-31

This document is the single entry point for the Low-Level Designs (LLDs) of every module in the Shomer.AI server and its first-party clients. It explains **how the modules are organized, why the codebase is shaped the way it is, and what rules every module must follow** so that any one of them can be replaced — OCR engine, classifier, Context Agent, notification channel, rate-limit store — without touching the others.

1. Index of module designs

#	Module	Design doc	One-line purpose
1	android_client	android_client/design.md	Kotlin / Compose app on the child's and parent's phones; UI, ViewModels, FCM, permissions.
2	sdk	sdk/design.md	Hand-written Kotlin SDK that every client uses to talk to the FastAPI server.
3	gatekeeper	gatekeeper/design.md	FastAPI middleware: rate-limit, trace-id, metrics, request-size and timeout enforcement.
4	ocr	ocr/design.md	Tesseract-based extraction of Hebrew + English text from chat screenshots.
5	classifier	classifier/design.md	DictaBERT-base frontline classifier (5-label offensive-Hebrew schema).
6	context_agent	context_agent/design.md	LLM-based contextual reasoner that resolves borderline classifications.
7	triage	triage/design.md	Deterministic decision gate between classifier and Context Agent — owns the A/B switch.
8	alerts	alerts/design.md	FCM push notifications to parents, idempotency, rate-limit, retry queue.
9	server	server/design.md	Integration view: FastAPI app, lifespan composition, cross-cutting concerns, NFR ownership.
10	audit_log	audit_log/design.md	Persistent record of every classification decision; 7-day retention; gold-label annotation surface for Meeting 8 ΔFPR evaluation.

2. Architectural principles — modular monolith + ports-and-adapters

2.1 Why a modular monolith

PRD §7.1 locks a **single-process FastAPI server** as the deployment target for the MVP. We chose the modular-monolith pattern (not microservices) for three reasons:

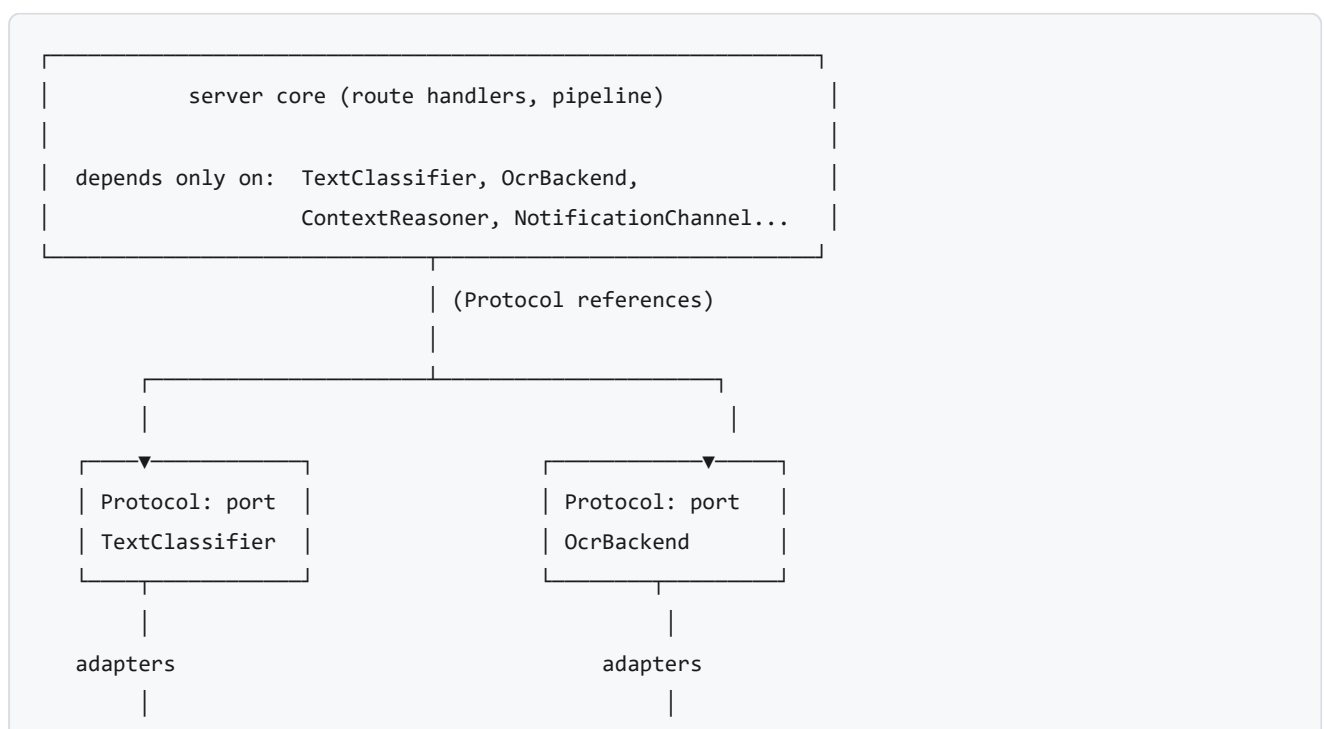
- **One household, one server.** The product runs on Alona's home server; horizontal scaling is not needed for the thesis SOM.
- **Debuggability.** A single Python process with one structured log stream and one in-process trace-id is dramatically easier to instrument and explain in an academic defense than a multi-service mesh.
- **Latency.** PRD §9 demands $p99 < 100$ ms for the frontline classifier path. Cross-process HTTP hops add 5–50 ms of latency we cannot afford.

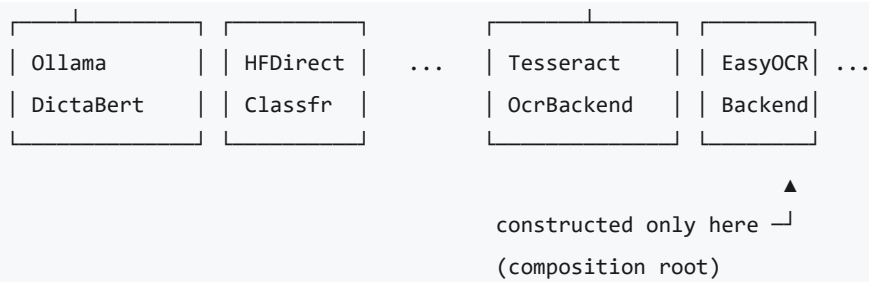
But "single process" must not mean "spaghetti." Every module is shaped so it could be lifted into its own process tomorrow, with no caller changes. That is the **ports-and-adapters** (hexagonal) layer of the design.

2.2 The rule

The server core depends on Protocols (interfaces), never on concrete adapter classes.

Each module exposes exactly **one port** (a `typing.Protocol` in Python, a Kotlin `interface` in the SDK / client) and ships **one or more adapters** that satisfy it. The server's composition root (`server/app/main.py lifespan()`) is the **only place** that imports concrete classes. Everything else types its dependencies by Protocol.





2.3 The five swap scenarios this design supports

Each of these is a **one-line change** in `server/app/main.py` `lifespan()`. No route handler, business logic, or other module changes.

(a) Swap Tesseract → EasyOCR

```
# Before:
ocr: OcrBackend = TesseractOcrBackend(settings.ocr)

# After:
ocr: OcrBackend = EasyOcrBackend(settings.ocr)
```

Use case: PRD §12 risk row — "Tesseract Hebrew CER > 15%" — fallback path.

(b) Swap DictaBERT (Ollama) → DictaBERT (HuggingFace direct) → DictaLM-2.0

```
# Before:
classifier: TextClassifier = OllamaDictaBertClassifier(settings.classifier)

# After (HuggingFace direct, no Ollama):
classifier: TextClassifier = HuggingFaceClassifier(settings.classifier)

# Or (PRD §12 risk row – DictaBERT-base won't cross F1 0.78):
classifier: TextClassifier = DictaLm2Classifier(settings.classifier)
```

(c) Swap GPT-4o-mini → Claude Haiku 4.5 → local Qwen; or swap the whole agent for a rule engine

The Context Agent has **two** ports — letting you swap the LLM provider *inside* the agent, OR replace the entire agent with a deterministic alternative.

```
# Swap the LLM the agent uses:
llm: LlmClient = GptMiniClient(settings.llm)
# llm: LlmClient = HaikuClient(settings.llm)
# llm: LlmClient = LocalQwenClient(settings.llm)
context_agent: ContextReasoner = LlmContextAgent(llm, ...)

# Or replace the entire reasoner with a deterministic rule engine:
# context_agent: ContextReasoner = RuleBasedReasoner(...)
```

(d) Swap FCM → APNs/SMS/Webhook

```
# Before:
notifier: NotificationChannel = FcmNotifier(settings.alerts)
# After:
notifier: NotificationChannel = SmsNotifier(settings.alerts)
# Or for a 3rd-party integration (school counselor dashboard):
notifier: NotificationChannel = WebhookNotifier(settings.alerts)
```

(e) Swap in-memory rate-limit store → Redis

```
# Before:
rate_store: RateLimitStore = InMemoryRateLimitStore()
# After:
rate_store: RateLimitStore = RedisRateLimitStore(settings.gatekeeper.redis_url)
```

Use case: when the server moves from in-memory state to a multi-worker / multi-replica deployment.

(f) Swap SQLite audit store → Postgres for scale-up

```
# Before (single household, local SQLite file):
audit: AuditStore = SqliteAuditStore(settings.audit_log)
# After (multi-tenant deployment, centralised Postgres):
audit: AuditStore = PostgresAuditStore(settings.audit_log)
```

Use case: when Shomer.AI moves beyond a single home server to a multi-tenant service, a centralised Postgres store enables cross-household analytics while keeping the same `AuditStore` Protocol contract. Zero changes to route handlers, `ClassificationPipeline`, Context Agent `ToolRunner`, or any test that types its dependency as `AuditStore`.

3. OOP discipline rules

Five hard rules every module follows. These are enforceable via `ruff` / `mypy` lints and code review.

Rule 1 — Depend on abstractions (Protocols), not concretions

Module boundaries are crossed only via Protocols. A consumer module never imports a concrete adapter class from another module — only the Protocol that adapter satisfies.

```
# WRONG — server/app/main.py imports a concrete adapter as a type
from .ocr.tesseract import TesseractOcrBackend
```

```
def handle_image(ocr: TesseractOcrBackend, ...): ...

# RIGHT — main.py types by Protocol; the concrete is constructed only in lifespan()
from .ocr import OcrBackend
def handle_image(ocr: OcrBackend, ...): ...
```

Rule 2 — Single responsibility per class

Each class owns one cohesive concern. A class that exceeds ~300 lines of code is a signal to split (e.g., `OcrBackend` was split into `OcrBackend` + `ImagePreprocessor` + `TesseractRunner` ; `ContextAgent` was split into `ContextAgent` + `LlmRouter` + `ToolRunner` + `TokenManager`).

Rule 3 — Open-closed

New adapters are added without modifying existing code. To support EasyOCR, you create `server/app/ocr/easyocr.py` containing `EasyOcrBackend(OcrBackend)` ; you do **not** edit `tesseract.py` , the strategy router, or the route handler.

Rule 4 — Liskov substitution (enforced by contract tests)

Every adapter must pass the same parametrized pytest suite as every other adapter for the same Protocol. The Protocol contract is therefore enforced at test time, not just at type-check time. See §5 below.

Rule 5 — Composition over inheritance

No inheritance chains deeper than one level. Reuse happens via composition + injection (e.g., `OllamaDictaBertClassifier` is composed with an `OllamaClient` and a `ConfidenceCalibrator` ; it does **not** subclass them).

4. Composition root pattern

`server/app/main.py` is the **only** place in the codebase that imports concrete adapter classes. The `lifespan()` async context manager constructs each adapter exactly once and stores it on `app.state` typed by its Protocol. Swapping an adapter is one edited line.

```
# server/app/main.py

from contextlib import asynccontextmanager
from fastapi import FastAPI

# Protocols (the only cross-module symbols imported by main.py except inside lifespan)
from .ocr import OcrBackend
```

```

from .classifier import TextClassifier
from .triage import TriageEngine
from .context_agent import ContextReasoner, LlmClient, TokenBudgetGuard
from .alerts import NotificationChannel, AlertRateLimiter
from .gatekeeper import RateLimitStore, TraceIdGenerator, MetricsEmitter

# Concrete adapters – imported ONLY here, inside the composition root
from .ocr.tesseract import TesseractOcrBackend
# from .ocr.easyocr import EasyOcrBackend # alt
from .classifier.ollama import OllamaDictaBertClassifier
# from .classifier.hf import HuggingFaceClassifier # alt
from .triage.rule_based import RuleBasedTriage
from .context_agent.agent import LlmContextAgent
from .context_agent.llm import GptMiniClient
# from .context_agent.llm import HaikuClient # alt
from .context_agent.token_manager import SqliteTokenManager
from .alerts.fcm import FcmNotifier
from .alerts.rate_limiter import InMemoryAlertRateLimiter
from .gatekeeper.rate_limit import InMemoryRateLimitStore
# from .gatekeeper.rate_limit import RedisRateLimitStore # alt
from .gatekeeper.trace_id import Uuid4TraceIdGenerator
from .gatekeeper.metrics import PrometheusMetricsEmitter

from .pipeline import ClassificationPipeline
from .settings import ServerSettings

@asynccontextmanager
async def lifespan(app: FastAPI):
    settings = ServerSettings()

    # — Every concrete adapter is constructed HERE and only HERE. —————
    # Swapping an adapter = changing one line below.

    ocr: OcrBackend = TesseractOcrBackend(settings.ocr)
    # ocr: OcrBackend = EasyOcrBackend(settings.ocr) # ← swap example

    classifier: TextClassifier = OllamaDictaBertClassifier(settings.classifier)
    # classifier: TextClassifier = HuggingFaceClassifier(settings.classifier) # ← swap

    triage: TriageEngine = RuleBasedTriage(settings.triage)

    llm: LlmClient = GptMiniClient(settings.llm)
    # llm: LlmClient = HaikuClient(settings.llm) # ← swap example
    budget: TokenBudgetGuard = SqliteTokenManager(settings.context_agent)
    context_agent: ContextReasoner = LlmContextAgent(llm, budget, settings.context_agent)
    # context_agent: ContextReasoner = RuleBasedReasoner(...) # ← swap whole agent

    rate_limiter: AlertRateLimiter = InMemoryAlertRateLimiter(settings.alerts)

```

```

notifier: NotificationChannel = FcmNotifier(settings.alerts, rate_limiter)
# notifier: NotificationChannel = SmsNotifier(settings.alerts, rate_limiter) # ← swap

rate_store: RateLimitStore = InMemoryRateLimitStore()
# rate_store: RateLimitStore = RedisRateLimitStore(settings.gatekeeper.redis_url)
trace_id_gen: TraceIdGenerator = Uuid4TraceIdGenerator()
metrics: MetricsEmitter = PrometheusMetricsEmitter(settings.gatekeeper)

# Pipeline is composed of Protocol-typed dependencies.
app.state.pipeline = ClassificationPipeline(
    ocr=ocr,
    classifier=classifier,
    triage=triage,
    context_agent=context_agent,
    notifier=notifier,
    rate_store=rate_store,
    trace_id_gen=trace_id_gen,
    metrics=metrics,
)

yield # – server is running –

# Shutdown
await app.state.pipeline.aclose()

```

Why this is the entire story: `ClassificationPipeline.__init__` takes Protocol-typed parameters. It never knows whether the OCR is Tesseract or EasyOCR; whether the classifier is Ollama-backed or HuggingFace-direct; whether the reasoner is an LLM agent or a rule engine. That ignorance is the design.

5. Contract testing

Each Protocol gets a **parametrized pytest contract suite** under `tests/contracts/`. Every adapter for that Protocol is parametrized through the same suite and must pass it. This is how we enforce Liskov substitution operationally — not just by type-checking, but by behaviour-checking.

```

tests/contracts/
├─ test_ocr_backend_contract.py      # parametrized over: TesseractOcrBackend, EasyOcrBackend, StubOcrBackend
├─ test_text_classifier_contract.py # parametrized over: OllamaDictaBertClassifier, HuggingFaceClassifier
├─ test_context_reasoner_contract.py # parametrized over: LlmContextAgent, RuleBasedReasoner, StubReasoner
├─ test_notification_channel_contract.py # parametrized over: FcmNotifier, SmsNotifier, EmailNotifier
└─ test_rate_limit_store_contract.py # parametrized over: InMemoryRateLimitStore, RedisRateLimitStore

```

Each contract test verifies the Protocol's invariants: - **Output schema:** result type matches the Protocol's declared return. - **Latency budget:** call returns within the documented p99 ceiling. - **Idempotency** where applicable (e.g., `alerts.send_alert` with the same `alert_id` produces the same `AlertResult` — no duplicate FCM messages). - **Error semantics:** documented failure modes return the documented fallback (e.g., `OcrBackend.process` returns `image_unreadable=True`, not raise, on a blank image).

A new adapter is "done" only when it passes the contract suite. Adding `EasyOcrBackend` does not require writing new tests — only registering it as a parametrize case.

6. Module dependency rules

The import graph is **acyclic**, and cross-module imports go through each module's `__init__.py` which re-exports **only the Protocol(s)**. Concrete adapter classes live in private submodules (`ocr/tesseract.py` , `ocr/easyocr.py`) and are imported only by the composition root.

Module	MAY import from	MUST NOT import from
ocr	stdlib, pytesseract, PIL, numpy, cv2	classifier, triage, context_agent, alerts, server.app.main
classifier	stdlib, ollama_client, transformers, torch, prompt.py	ocr, triage, context_agent, alerts, server.app.main
trriage	classifier Protocol only (NOT the concrete OllamaDictaBertClassifier)	any concrete adapter from another module
context_agent	AuditStore Protocol (for read_conversation_history tool), LLM SDK Protocols, token-manager Protocol, tool runner	classifier impl, ocr impl, alerts impl, SqliteAuditStore (only the Protocol)
alerts	NotificationChannel Protocol, AlertRateLimiter Protocol, firebase-admin	classifier, ocr, context_agent, triage
gatekeeper	structlog, prometheus_client, slowapi	any business module (classifier, ocr, context_agent, alerts, triage, audit_log)
audit_log	stdlib, aiosqlite, structlog, prometheus_client	classifier, ocr, triage, context_agent, alerts, gatekeeper, server.app.main
server (route handlers, pipeline)	EVERY module's PROTOCOL (re-exported from each module's __init__.py)	concrete adapter classes (except inside main.py lifespan — see Rule below)

The composition-root exception: `server/app/main.py` is the *one* file allowed to import concrete adapter classes. This is by design: it is the single point where DI happens. A lint rule (`import-linter`, contract: `forbidden`) enforces this. New developers do not get to add a concrete import in a route handler or pipeline class — only inside `lifespan()`.

Re-export convention: each module's `__init__.py` looks like:

```
# server/app/ocr/__init__.py
from .protocol import OcrBackend, OcrResult
__all__ = ["OcrBackend", "OcrResult"]
# Concrete adapters (TesseractOcrBackend, EasyOcrBackend, ...) are NOT re-exported.
# They are imported by full path inside server/app/main.py's lifespan() only.
```

This guarantees the rule via tooling: anyone writing `from app.ocr import TesseractOcrBackend` outside `main.py` gets a flake / `import-linter` failure.

7. Reading order for a new contributor

A new contributor onboarding to Shomer.AI should read these documents in this order:

1. **This README** (5 minutes) — understand the modular-monolith + ports-and-adapters story.
2. `server/design.md` — the integration view: how all the modules wire together at startup.
3. **The module they are about to work on** — its LLD §2 (port), §2.5 (isolation guarantees), §3 (internal design).
4. **The contract test** for their Protocol — to understand what their adapter must guarantee.

That is the entire architecture brief.